DSA0601 – DATA HANDLING AND VISUALIZATION

ACTIVITY -3

192424221 – Bale Naveen Kumar Yadav

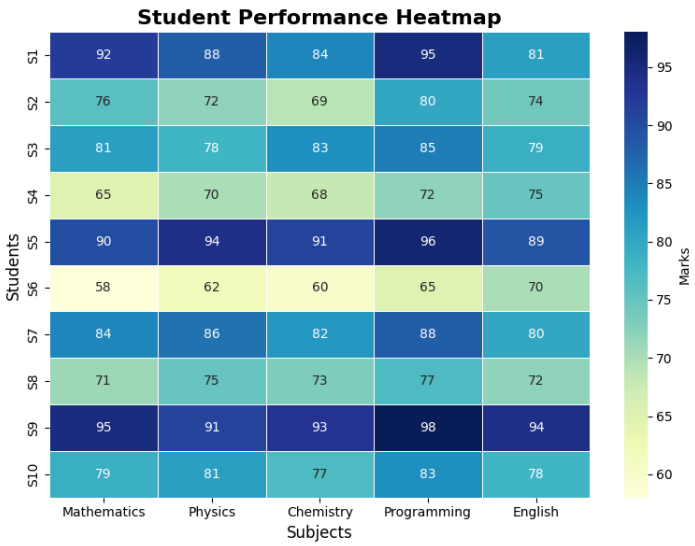
Visualization Activities with Datasets

Activity 1: Heatmap – Student Performance Analysis

Student	Mathematics	Physics	Chemistry	Programming	English
S1	92	88	84	95	81
S2	76	72	69	80	74
S3	81	78	83	85	79
S4	65	70	68	72	75
S5	90	94	91	96	89
S6	58	62	60	65	70
S7	84	86	82	88	80
S8	71	75	73	77	72
S9	95	91	93	98	94
S10	79	81	77	83	78

1.

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Student performance data
data = {
    'Mathematics': [92, 76, 81, 65, 90, 58, 84, 71, 95, 79],
    'Physics': [88, 72, 78, 70, 94, 62, 86, 75, 91, 81],
    'Chemistry': [84, 69, 83, 68, 91, 60, 82, 73, 93, 77],
    'Programming': [95, 80, 85, 72, 96, 65, 88, 77, 98, 83],
    'English': [81, 74, 79, 75, 89, 70, 80, 72, 94, 78]
}

# Student names
students = ['S1', 'S2', 'S3', 'S4', 'S5',
            'S6', 'S7', 'S8', 'S9', 'S10']

# Create DataFrame
df = pd.DataFrame(data, index=students)

# Set figure size
plt.figure(figsize=(8, 6))

# Draw heatmap
sns.heatmap(
    df,
    annot=True,          # Display values
    fmt='d',             # Integer format
    cmap='YlGnBu',       # Color palette
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Marks'}
)

# Add title and labels
plt.title('Student Performance Heatmap', fontsize=16, fontweight='bold')
plt.xlabel('Subjects', fontsize=12)
plt.ylabel('Students', fontsize=12)

# Display the heatmap
plt.tight_layout()
plt.show()
```



2. Highest Overall Performer: S9 (Average = 94.2)

3. Highest Average Subject: Programming (Average = 83.9)

4. S6 (Average = 63.0) – Needs support in all subjects.

5. Clearly highlights high and low scores using colours.

Makes it easy to compare students and subjects.

Helps teachers quickly identify top performers and students needing support.

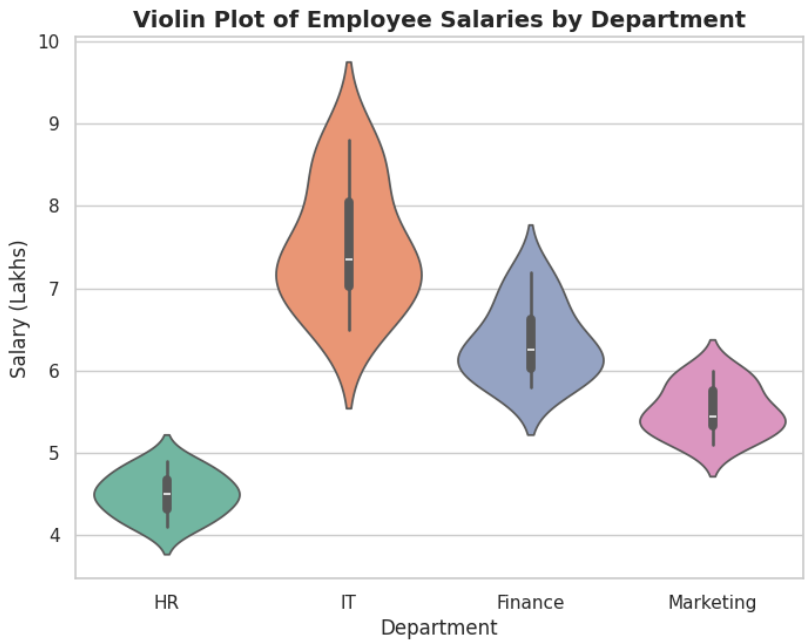
Reveals subject-wise performance trends.

Activity 2: Violin Plot & Boxplot – Employee Salary Analysis

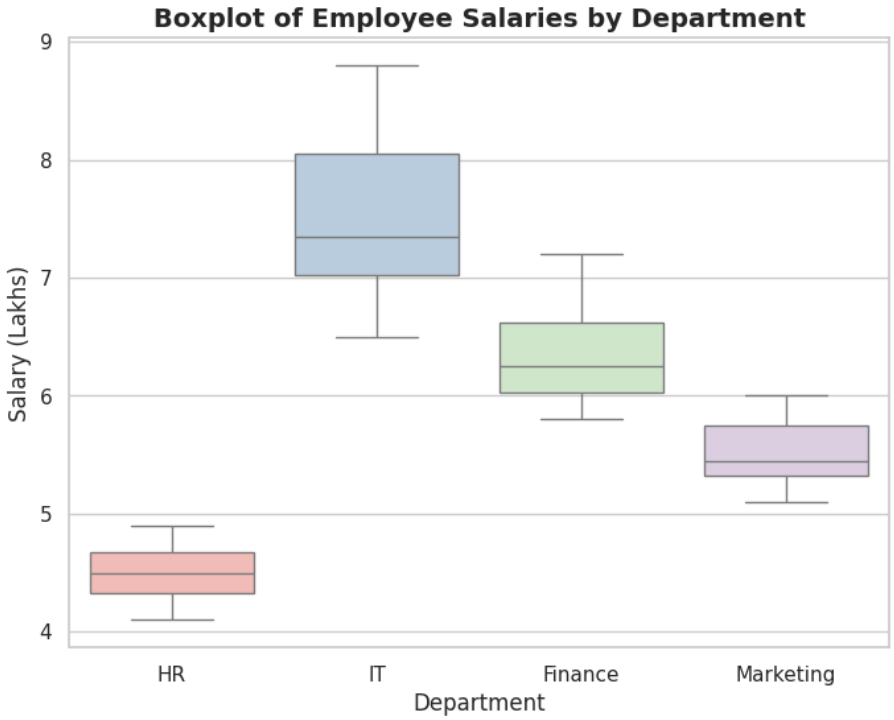
Employee	HR	IT	Finance	Marketing
E1	4.2	6.5	5.8	5.1
E2	4.5	7.0	6.2	5.3
E3	4.7	7.4	6.0	5.5
E4	4.1	6.8	5.9	5.4
E5	4.6	8.2	6.7	5.8
E6	4.3	7.6	6.4	5.6
E7	4.8	8.5	6.9	5.9
E8	4.4	7.1	6.1	5.2
E9	4.9	8.8	7.2	6.0
E10	4.5	7.3	6.3	5.4

1.

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Employee Salary Data (in Lakhs)
data = {
    'Employee': ['E1','E2','E3','E4','E5','E6','E7','E8','E9','E10'],
    'HR': [4.2, 4.5, 4.7, 4.1, 4.6, 4.3, 4.8, 4.4, 4.9, 4.5],
    'IT': [6.5, 7.0, 7.4, 6.8, 8.2, 7.6, 8.5, 7.1, 8.8, 7.3],
    'Finance': [5.8, 6.2, 6.0, 5.9, 6.7, 6.4, 6.9, 6.1, 7.2, 6.3],
    'Marketing': [5.1, 5.3, 5.5, 5.4, 5.8, 5.6, 5.9, 5.2, 6.0, 5.4]
}

# Create DataFrame
df = pd.DataFrame(data)

# Convert data into long format for Seaborn
salary_data = df.melt(id_vars='Employee',
                      var_name='Department',
                      value_name='Salary')

# Set plot style
sns.set(style="whitegrid")

# ------------------------------
# Violin Plot
# ------------------------------
plt.figure(figsize=(8,6))
sns.violinplot(x='Department',
               y='Salary',
               data=salary_data,
               palette='Set2')

plt.title('Violin Plot of Employee Salaries by Department',
          fontsize=14,
          fontweight='bold')
plt.xlabel('Department')
plt.ylabel('Salary (Lakhs)')
plt.show()

# ------------------------------
# Box Plot
# ------------------------------
plt.figure(figsize=(8,6))
sns.boxplot(x='Department',
            y='Salary',
            data=salary_data,
            palette='Pastel1')

plt.title("Boxplot of Employee Salaries by Department",
          fontsize=14,
          fontweight='bold')
plt.xlabel('Department')
plt.ylabel('Salary (Lakhs)')
plt.show()
```



2.



3. IT has the largest variation.

4. IT has the highest median salary.

5. The Violin Plot better represents the overall distribution.

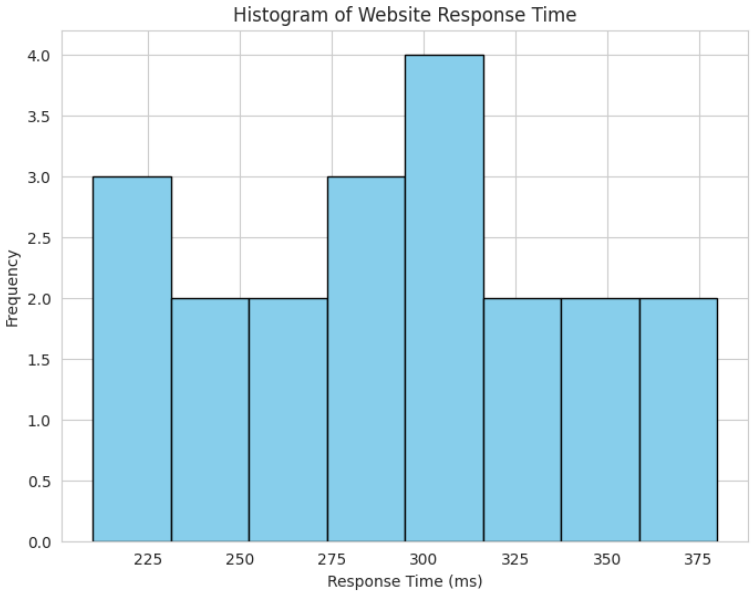
Activity 3: Histogram & Density Plot – Website Response Time

Obs	Response Time (ms)
1	210
2	225
3	230
4	240
5	250
6	260
7	270
8	280
9	285
10	290
11	300
12	305
13	310
14	315
15	320
16	330
17	340
18	350
19	360
20	380

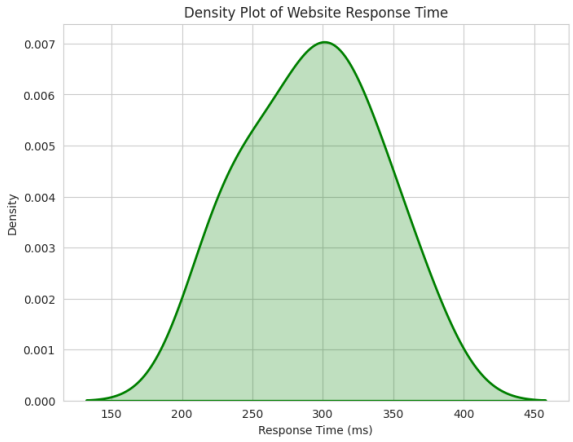
Questions:

1.

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Website Response Time Data (ms)
response_time = [
    210, 225, 230, 240, 250,
    260, 270, 280, 285, 290,
    300, 305, 310, 315, 320,
    330, 340, 350, 360, 380
]

# Create DataFrame
df = pd.DataFrame({
    'Response Time (ms)': response_time
})

# Set plot style
sns.set_style("whitegrid")

# ----------------------------------------
# Histogram
# ----------------------------------------
plt.figure(figsize=(8,6))

plt.hist(
    df['Response Time (ms)'],
    bins=8,
    color='skyblue',
    edgecolor='black'
)

plt.title('Histogram of Website Response Time')
plt.xlabel('Response Time (ms)')
plt.ylabel('Frequency')

plt.show()

# ----------------------------------------
# Density Plot (KDE)
# ----------------------------------------
plt.figure(figsize=(8,6))

sns.kdeplot(
    df['Response Time (ms)'],
    fill=True,
    color='green',
    linewidth=2
)

plt.title('Density Plot of Website Response Time')
plt.xlabel('Response Time (ms)')
plt.ylabel('Density')

plt.show()
```



2.



3. The distribution is **slightly positively (right) skewed**, as the tail extends toward higher response times.

4. The **common range** of website response times is **250–330 ms**.

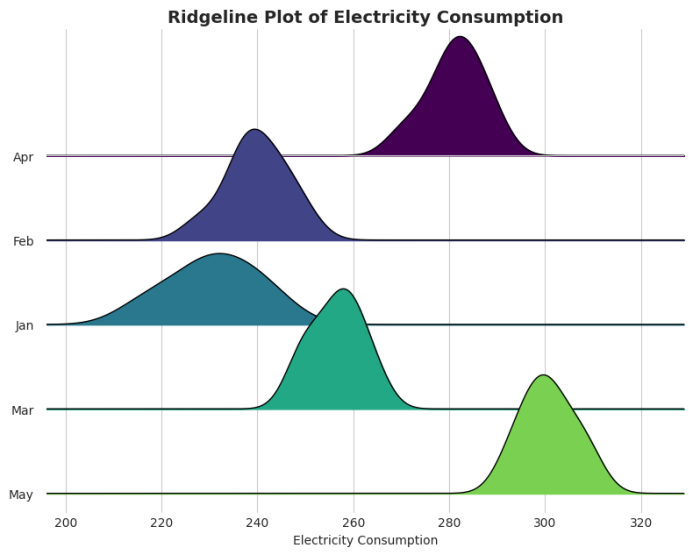
5. The **Density Plot** better represents the distribution because it clearly shows the overall shape and skewness of the response time data. However, the **Histogram** is better for displaying the actual frequencies of observations.

Activity 4: Ridgeline Plot – Electricity Consumption

House	Jan	Feb	Mar	Apr	May
H1	220	235	250	275	295
H2	240	245	260	285	305
H3	230	238	255	280	300
H4	215	228	248	270	292
H5	225	240	258	282	298
H6	245	250	265	290	310
H7	235	242	259	284	302
H8	228	236	252	278	297
H9	238	246	262	287	307
H10	232	239	256	281	301

Questions:

1.
```
# Install joypy (run once)
!pip install joypy

# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import joypy

# Electricity Consumption Data
data = {
    'House': ['H1', 'H2', 'H3', 'H4', 'H5', 'H6', 'H7', 'H8', 'H9', 'H10'],
    'Jan': [220, 240, 230, 215, 225, 245, 235, 228, 238, 232],
    'Feb': [235, 245, 238, 228, 240, 250, 242, 236, 246, 239],
    'Mar': [250, 260, 255, 248, 258, 265, 259, 252, 262, 256],
    'Apr': [275, 285, 280, 270, 282, 290, 284, 278, 287, 281],
    'May': [295, 305, 300, 292, 298, 310, 302, 297, 307, 301]
}

# Create DataFrame
df = pd.DataFrame(data)

# Convert to long format
long_df = df.melt(
    id_vars='House',
    var_name='Month',
    value_name='Consumption'
)

# Create Ridgeline Plot
fig, axes = joypy.joyplot(
    long_df,
    by='Month',
    column='Consumption',
    figsize=(8, 6),
    colormap=plt.cm.viridis,
    linewidth=1,
    overlap=1,
    grid=True
)

plt.title("Ridgeline Plot of Electricity Consumption", fontsize=14, fontweight='bold')
plt.xlabel("Electricity Consumption")
plt.show()
```



2. **May** has the **highest average electricity consumption** (**300.7 units**).

3. **January** has the **widest spread** with a range of **30 units**.

4. There is a **steady increasing trend** in electricity consumption from **January to May**.

5. A **ridgeline plot** effectively compares the distribution and trends of electricity consumption across multiple months in a single visualization.

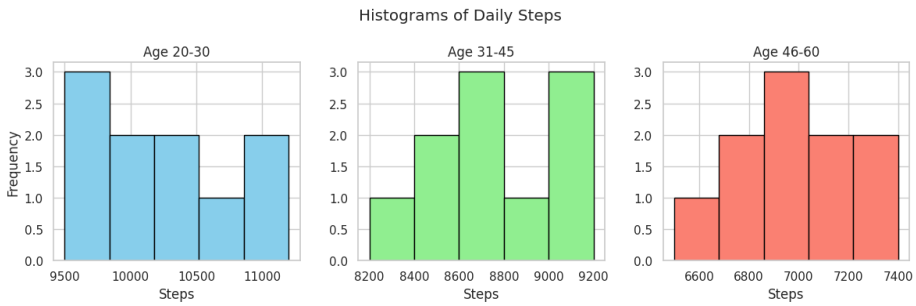
Activity 5: Histogram, Density & Boxplot – Daily Steps

Person	20–30	31–45	46–60
P1	9500	8200	6500
P2	10200	8600	6700
P3	9800	8400	6900
P4	11000	9000	7200
P5	10500	8900	7000
P6	9700	8500	6800
P7	10100	8700	7100
P8	10800	9100	7300
P9	11200	9200	7400
P10	9900	8600	6950

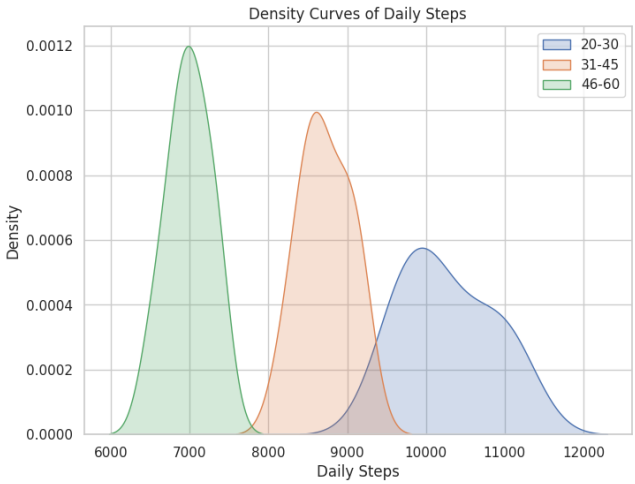
Questions:

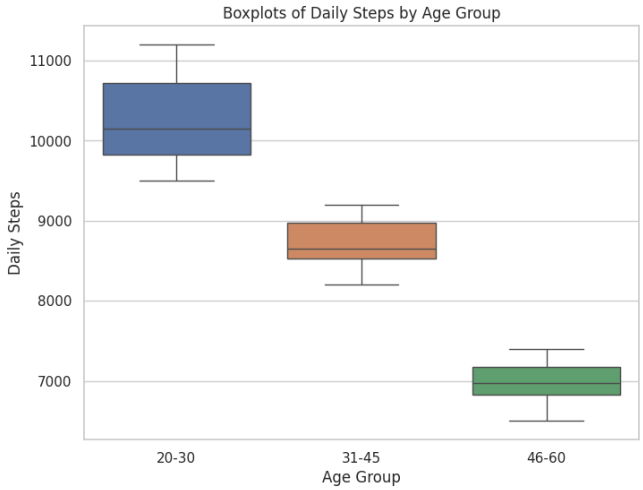
1.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Daily Steps Data
data = {
    '20-30': [9500, 10200, 9800, 11000, 10500, 9700, 10100, 10800, 11200, 9900],
    '31-45': [8200, 8600, 8400, 9000, 8900, 8500, 8700, 9100, 9200, 8600],
    '46-60': [6500, 6700, 6900, 7200, 7000, 6800, 7100, 7300, 7400, 6950]
}

# Create DataFrame
df = pd.DataFrame(data)

# Set plot style
sns.set(style="whitegrid")

# ------------------------------
# 1. Histograms
# ------------------------------
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.hist(df['20-30'], bins=5, color='skyblue', edgecolor='black')
plt.title('Age 20-30')
plt.xlabel('Steps')
plt.ylabel('Frequency')

plt.subplot(1,3,2)
plt.hist(df['31-45'], bins=5, color='lightgreen', edgecolor='black')
plt.title('Age 31-45')
plt.xlabel('Steps')

plt.subplot(1,3,3)
plt.hist(df['46-60'], bins=5, color='salmon', edgecolor='black')
plt.title('Age 46-60')
plt.xlabel('Steps')

plt.suptitle('Histograms of Daily Steps')
plt.tight_layout()
plt.show()

# ------------------------------
# 2. Density Curves
# ------------------------------
plt.figure(figsize=(8,6))

sns.kdeplot(df['20-30'], label='20-30', fill=True)
sns.kdeplot(df['31-45'], label='31-45', fill=True)
sns.kdeplot(df['46-60'], label='46-60', fill=True)

plt.title('Density Curves of Daily Steps')
plt.xlabel('Daily Steps')
plt.ylabel('Density')
plt.legend()
plt.show()

# ------------------------------
# 3. Boxplots
# ------------------------------
plt.figure(figsize=(8,6))

sns.boxplot(data=df)

plt.title('Boxplots of Daily Steps by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Daily Steps')

plt.show()
```



2.



3. 

4. The **20–30 age group** has the **highest median daily steps**, with a median of **10,150 steps**.

5. The **boxplot** is the best visualization because it clearly compares the **median, spread, and potential outliers** of daily steps across the three age groups.